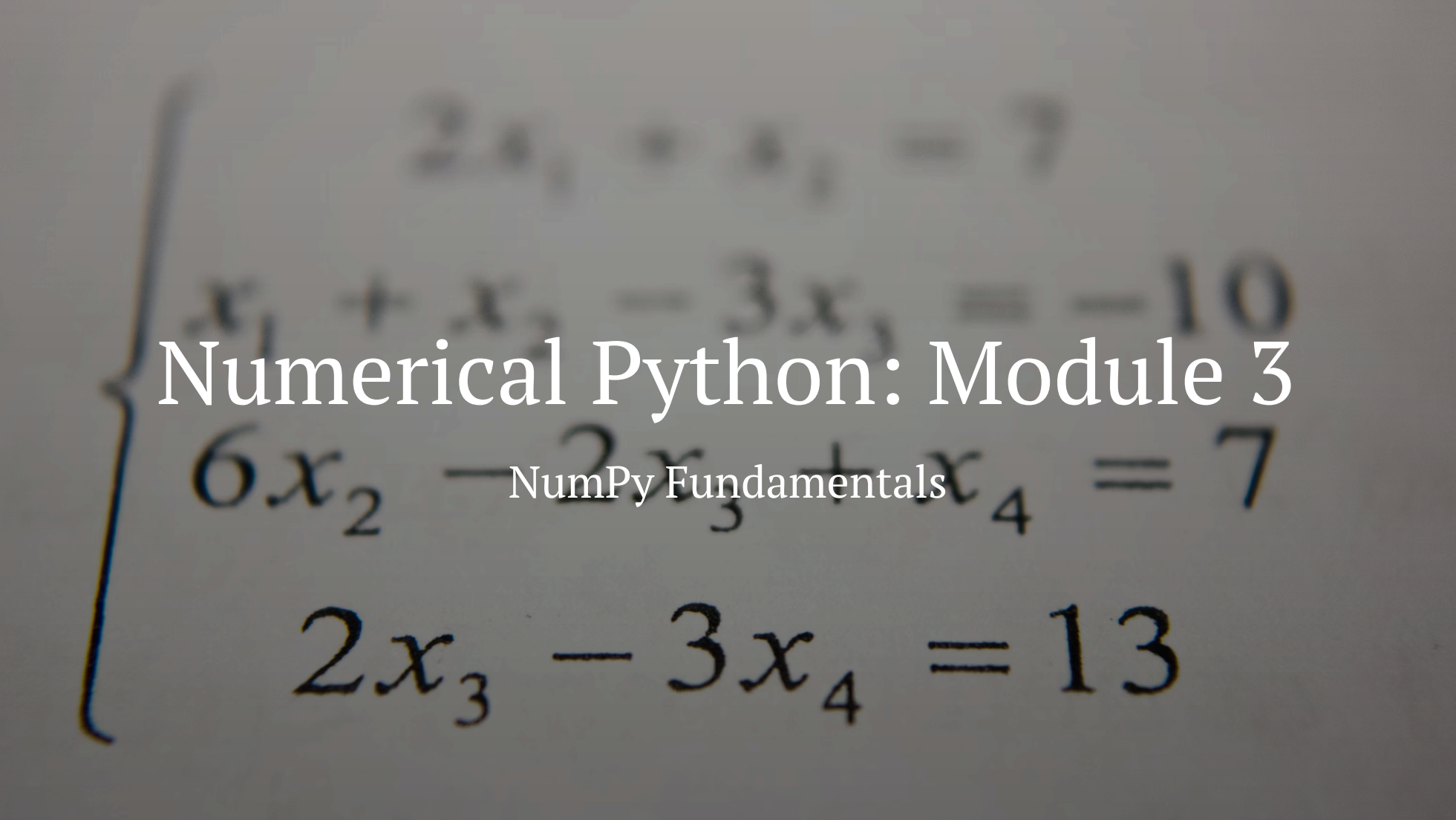
Numerical Python: Module 3

NumPy Fundamentals



1. What is NumPy?

NumPy (Numerical Python) is the fundamental package for scientific computing in Python. It provides a high-performance multidimensional **array** object and tools for working with these arrays.

- **ndarray:** The core data structure, a fast and flexible container for large datasets.
- **Vectorization:** Expressing operations as occurring on entire arrays rather than individual elements.

2. Why Use NumPy over Python Lists?

NumPy arrays are faster and more memory-efficient than standard Python lists for numerical data.

1. **Fixed Type:** Every element in a NumPy array must be of the same data type, allowing for optimized memory storage.
2. **Contiguous Memory:** Arrays are stored in one continuous block of memory, enabling faster access (SIMD).
3. **No Type Checking:** Operations don’t require Python’s overhead of checking types for every single element during a loop.

Basically… It's C-speed performance with Python-style syntax!

3. Creating Arrays

From Lists

```python
import numpy as np

# 1D Array
a = np.array([1, 2, 3])

# 2D Array (Matrix)
b = np.array([[1, 2], [3, 4]])
```

Built-in Functions

```python
# Array of zeros
zeros = np.zeros((2, 3))

# Linear spacing
# (start, stop, num_elements)
lin = np.linspace(0, 10, 5)
```

4. Array Attributes & Inspection

Understanding the structure of your data is critical in data science.

```python
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Dimensions (rows, columns)
print(arr.shape)  # Output: (2, 3)

# Number of dimensions
print(arr.ndim)   # Output: 2

# Data type of elements
print(arr.dtype)  # Output: int64 (or int32)
```

5. Basic Operations (Vectorization)

Operations are applied **element-wise**. No explicit `for` loops required!

```python
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

# Arithmetic
print(a + b)  # [11, 22, 33]
print(a * b)  # [10, 40, 90]

# Scalar operations
print(a * 2)  # [20, 40, 60]
```

```python
# Comparison
print(a > 15) # [False, True, True]
```

6. Slicing and Indexing

NumPy slicing is similar to Python lists but extends to multiple dimensions.

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Access specific element [row, col]
print(arr[0, 1])      # Output: 2

# Slice rows [start:stop]
print(arr[0:2, :])    # First two rows

# Slice columns
print(arr[:, 1])      # All rows, second column
```

7. Advanced Array Operations

NumPy provides powerful built-in methods to summarize and restructure data without manual loops.

7.1 Statistical Functions

You can perform calculations across the entire array or specific axes (0 for columns, 1 for rows).

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Sum of all elements
print(np.sum(arr))    # Output: 21

# Mean (Average)
print(np.mean(arr))   # Output: 3.5

# Standard Deviation
print(np.std(arr))    # Output: 1.707...
```

7.2 Reshaping and Transposing

Change the structure of your data without changing the data itself.

```
data = np.arange(6) # [0, 1, 2, 3, 4, 5]

# Change to 2 rows, 3 columns
reshaped = data.reshape((2, 3))
print(reshaped)

# Transpose (Swap rows and columns)
print(reshaped.T)

# Flatten back to 1D
print(reshaped.flatten())
```

7.3 Universal Functions (ufuncs)

These functions operate element-wise on arrays, providing "vectorized" wrappers for common math operations.

```
arr = np.array([0, 1, 4, 9])

# Square root
print(np.sqrt(arr))   # [0., 1., 2., 3.]

# Exponential (e^x)
print(np.exp(arr))

# Trigonometric
print(np.sin(arr))
```

8. Summary Comparison

Feature	Python List	NumPy Array
Data Types	Mixed types allowed	Single type only (homogeneous)
Performance	Slower for math	Extremely fast (C-based)
Memory	Higher overhead	Optimized & Contiguous
Functionality	General purpose	Specialized for Math/Science

End

Reference Sources:

- NumPy Quickstart Tutorial
- Real Python: Look at NumPy